

LexLens Build Spec 01: GhaLII Mirror and Reference Resolver

For. Codex, or any AI coding assistant working on the LexLens repo. **Owner.** Owura Bediako. **Date.** 2026-05-26. **Status.** Ready to build. **Estimated effort.** Two to three days of focused work.

Read `CLAUDE.md` in the repo root before starting. The writing rules and the hard rules on hallucination apply to all code comments and docstrings produced under this spec.

1. What this spec covers

Two coherent components, built together because the second depends on the first.

Component A: GhaLII mirror. A one-time downloader that pulls the 1992 Constitution and every Act of Parliament currently in force from ghalii.org. Stores them locally as raw HTML and parsed JSON. Refreshable on a schedule.

Component B: Reference resolver. A library that takes a string like "Act 769" or "the National Communications Authority Act, 2008" and returns the local file path, parsed structure, and a specific section if requested. Used by the bill parser to link clauses in the NITA Bill to the Acts they reference.

Out of scope for this spec. The bill parser itself. The risk classifier. The constitutional check. The frontend. The API. Those come in later specs.

2. Vocabulary

The terms matter. Mixing them will confuse legal reviewers.

- **Constitution.** The 1992 Constitution of Ghana. One document. Top of the legal hierarchy. Chunked by article.
- **Act of Parliament.** A statute passed by Parliament. Each Act has a number (Act 769, Act 843). There are hundreds. Stored separately from the Constitution.
- **Bill.** A proposed law, not yet passed. The thing LexLens analyses. Stored under

docs/bills/ .

- **Clause.** A unit within a Bill (clause 12, clause 12(3), clause 12(3)(b)).
- **Section.** A unit within an Act (section 12, section 12(3)). When a Bill is passed, its clauses become sections.
- **Article.** A unit within the Constitution (Article 19, Article 19(2)(d)).

Use these terms exactly. Do not write "act of the Constitution" anywhere.

3. Source URLs

GhaLII is at <https://ghalii.org>. The relevant endpoints are below. Verify each one with a manual browser visit before relying on it. URL structures may have changed since this spec was written.

- Constitution landing page. <https://ghalii.org/akn/gh/act/1992/constitution>
- All current legislation listing. <https://ghalii.org/legislation/>
- Individual Act page pattern. https://ghalii.org/akn/gh/act/{year}/{act_number}
- Akoma Ntoso XML download pattern. Append `/eng@.xml` to the Act URL.

GhaLII uses the Akoma Ntoso XML standard for structured legal documents. Prefer the XML over the HTML wherever both are available. XML preserves the hierarchical structure (chapter, part, section, sub-section, paragraph) without needing to parse HTML divs.

4. Repo placement

All work under this spec lives in two locations.

<code>src/lexlens/reference/</code>	
<code>__init__.py</code>	
<code>mirror.py</code>	Downloader entry point
<code>ghalii_client.py</code>	HTTP client with politeness controls
<code>parser_xml.py</code>	Akoma Ntoso XML to internal JSON
<code>parser_html.py</code>	HTML fallback when XML is unavailable
<code>resolver.py</code>	Reference resolution library
<code>registry.py</code>	Index of mirrored documents
<code>schemas.py</code>	Pydantic models for Act, Section, Article

```
docs/reference/
  constitution_1992/
    raw.xml           Akoma Ntoso source
    parsed.json       Internal structured form
    articles/         One markdown file per article
      article_001.md
      article_019.md
      ...
  ghana_acts/
    registry.json     Index of all mirrored Acts
    act_0769_nca_2008/
      raw.xml
      parsed.json
      sections/
        section_001.md
        ...
    act_0843_data_protection_2012/
    act_1038_cybersecurity_2020/
    ...

scripts/
  mirror_ghalii.py    CLI entry point for the downloader
  refresh_mirror.py    Quarterly refresh script
```

Act folder names use zero-padded four-digit Act numbers (act_0769, not act_769) so directory listings sort correctly.

5. Component A: GhaLII Mirror

5.1 Behaviour

The mirror script does five things in order.

1. Fetches the Constitution as Akoma Ntoso XML. Parses to internal JSON. Writes article-level markdown files.
2. Fetches the full list of current Acts from the legislation index. Extracts Act number, year, title, and URL for each.
3. For each Act, fetches the XML if available, falls back to HTML if not. Parses to internal JSON. Writes section-level markdown files.
4. Writes a top-level `registry.json` listing every mirrored document with its metadata.
5. Logs every fetch, parse, and write to `logs/mirror_run_{timestamp}.jsonl`.

5.2 Politeness controls

These are non-negotiable. GhaLII is a small operation.

- One request per second. Use `time.sleep(1.0)` between fetches. Not negotiable.
- Set a clear user-agent. Format: `LexLens/0.1 (research mirror; contact: owura@govaxis.ai)` . Update the email to whatever Owura prefers before merging.
- Respect `robots.txt` . Parse it on first fetch using `urllib.robotparser` . Abort if the script's user-agent is disallowed for any URL it tries to access.
- Cache aggressively. Once a document is mirrored, do not refetch unless explicitly told to. The refresh script handles updates.
- Set a hard timeout of 30 seconds per request. Retry twice with exponential backoff (2s, 4s) on connection errors. Never retry on 4xx responses.
- If the script encounters more than five consecutive failures, abort. The site is likely under stress or has changed structure.

5.3 Idempotence

The script must be safely re-runnable. Specifically:

- If a target file already exists and the source has not changed (compare content hashes), skip the fetch.
- If a parse fails, write the raw file but mark the entry in the registry as `parse_failed: true` . Do not crash the whole run.
- A second run with no changes upstream should make zero HTTP requests and exit in under five seconds.

5.4 CLI

```
# Full mirror, first run
python scripts/mirror_ghalii.py --target all

# Just the Constitution
python scripts/mirror_ghalii.py --target constitution

# Just a specific Act by number
python scripts/mirror_ghalii.py --target act --number 769

# Refresh, only fetches documents updated since last run
python scripts/refresh_mirror.py
```

```
# Dry run, lists what would be fetched, makes no requests beyond the index
python scripts/mirror_ghalii.py --target all --dry-run
```

5.5 Internal JSON schema

Defined in `schemas.py` using Pydantic v2. Both Constitution and Acts use a shared `LegalDocument` base. The schemas below are the contract. Other modules depend on them.

```
class TextNode(BaseModel):
    """A single addressable unit of legal text (article, section, sub-section, pa
    id: str # Stable ID, e.g. "art_19_2_d" or "s_12_3"
    parent_id: Optional[str] # The containing node's ID, or None for top
    level: Literal["article", "section", "subsection", "paragraph", "subparagraph
    number: str # The displayed number, e.g. "19", "2", "d"
    heading: Optional[str] # The heading text if present
    text: str # The verbatim text content
    children_ids: List[str] # IDs of nested nodes in order

class LegalDocument(BaseModel):
    """A Constitution or an Act."""
    doc_type: Literal["constitution", "act"]
    act_number: Optional[int] # None for Constitution
    year: int
    title: str
    short_title: Optional[str]
    long_title: Optional[str]
    date_assented: Optional[date]
    date_commenced: Optional[date]
    last_amended: Optional[date]
    status: Literal["in_force", "repealed", "partially_repealed"]
    source_url: str
    content_hash: str # SHA-256 of the raw source, for change dete
    fetched_at: datetime
    nodes: Dict[str, TextNode] # All addressable nodes keyed by ID
    root_node_ids: List[str] # Top-level nodes in document order

class RegistryEntry(BaseModel):
    """One entry in the top-level registry of all mirrored documents."""
    doc_type: Literal["constitution", "act"]
    act_number: Optional[int]
    year: int
    title: str
    short_title: Optional[str]
    status: str
```

```

file_path: str # Relative to repo root
content_hash: str
fetched_at: datetime
parse_failed: bool = False
parse_error: Optional[str] = None

```

5.6 Parsing rules

For the XML parser. Akoma Ntoso is a verbose standard. Extract only what `LegalDocument` needs.

- Each `<section>`, `<subsection>`, `<paragraph>`, and `<subparagraph>` becomes a `TextNode`.
- `<heading>` content goes into the `heading` field, never into `text`.
- Text content is extracted from `<content><p>` elements, joined with single spaces, with whitespace normalised (no double spaces, no leading or trailing whitespace).
- Cross-references inside text (`<ref href="...">`) are preserved as text but the href is captured to a separate `references` list on the node. This is what the resolver will use later.
- The `id` field is derived from the Akoma Ntoso `eld` attribute where present, otherwise constructed from the hierarchy (`s_12_3` for section 12 subsection 3).

For the HTML fallback parser. Only used if the XML endpoint returns 404. Use BeautifulSoup with the lxml parser. The HTML structure on GhaLII is reasonably consistent for current legislation, but expect at least 5 percent of documents to have layout quirks. When the parser cannot confidently identify a section boundary, mark the document `parse_failed: true` and move on. Do not silently produce incomplete output.

5.7 Markdown output

For each `TextNode` at section or article level, write a markdown file. The format:

```

# Section 12. Heading text if present

(1) Sub-section text here.

(2) Another sub-section.
    (a) A paragraph.
    (b) Another paragraph.

```

These markdown files are for human review and for use as direct context in LLM prompts.

They are derived from `parsed.json` and should be regenerated whenever `parsed.json` changes. Do not edit them by hand.

6. Component B: Reference Resolver

6.1 Behaviour

The resolver answers four kinds of question.

1. Given a citation string, return the matching `LegalDocument`.
2. Given a citation string and a section reference, return the specific `TextNode`.
3. Given a document and a section ID, return the verbatim text of that section.
4. Given a Bill clause's extracted references, return a resolution report.

6.2 API

Defined in `resolver.py`. Stable surface. Other modules will depend on these signatures.

```
class ReferenceResolver:
    def __init__(self, registry_path: Path = Path("docs/reference/ghana_acts/regi
        ...

    def resolve_act(self, citation: str) -> Optional[LegalDocument]:
        """
        Resolve a citation like 'Act 769', 'National Communications Authority Act
        or 'Act 769 of 2008'. Returns the matched LegalDocument or None.
        Matching is case-insensitive and tolerant of common variations.
        """

    def resolve_section(self, citation: str, section_ref: str) -> Optional[TextNo
        """
        Resolve a section within an Act. section_ref like 's.12', 'section 12(3)'
        '12(3)(b)'. Returns the deepest matching TextNode or None.
        """

    def resolve_constitution_article(self, article_ref: str) -> Optional[TextNode
        """
        Resolve an article like 'Article 19', '19(2)(d)', 'Art. 125'. Returns
        the deepest matching TextNode in the Constitution or None.
        """

    def resolve_references(self, references: List[str]) -> ResolutionReport:
```

```

"""
Bulk resolve a list of citation strings. Returns a report with
resolved, unresolved, and ambiguous matches.
"""

class ResolutionReport(BaseModel):
    resolved: List[ResolvedReference]
    unresolved: List[str] # Original strings that could not be matched
    ambiguous: List[AmbiguousMatch] # Strings that matched more than one document

```

6.3 Citation parsing

This is where most reference resolution systems fail. The patterns to handle, with examples:

- "Act 769" → look up by act_number
- "Act 769 of 2008" → look up by act_number, verify year
- "the National Communications Authority Act, 2008" → look up by title, verify year
- "the National Communications Authority Act (Act 769)" → both signals available
- "NCA Act" → short title or alias, requires an aliases table
- "section 12 of Act 769" → resolve Act first, then section
- "section 12(3)(b) of the Data Protection Act" → resolve Act, then nested section
- "Article 19(2)(d) of the Constitution" → resolve Constitution article
- "the Constitution" alone → resolve to the Constitution document
- "Act 769, as amended" → resolve to Act 769, flag the amendment qualifier

Build a citation parser as a small grammar, not a pile of regexes. Use `pyparsing` or write a hand-rolled recursive descent parser. The parser returns a structured `Citation` object with fields for `act_number`, `year`, `title`, `section_ref`, `qualifier`. The resolver consumes the `Citation` object, not raw strings.

6.4 Aliases

Some Acts are universally known by abbreviation. Maintain a small YAML file at

```
src/lexlens/reference/aliases.yaml:
```

```

- act_number: 769
  aliases: ["NCA Act", "Communications Authority Act"]
- act_number: 843

```

```
aliases: ["DPA", "Data Protection Act"]
- act_number: 1038
aliases: ["Cybersecurity Act"]
```

The resolver loads this on init and tries alias matching after exact and fuzzy title matching has failed.

6.5 Ambiguity handling

Never silently pick one when multiple match. If “Act 769” exists in 1965 and 2008 (rare but possible across the full corpus), the resolver returns `AmbiguousMatch` and the caller decides. Logged at WARNING level so the calling code can surface it to the analyst.

7. Testing

Tests live under `tests/reference/`. Use pytest. Three test classes.

7.1 Unit tests for parsing

Hand-craft three minimal Akoma Ntoso XML fixtures under `tests/reference/fixtures/xml/` and three HTML fixtures under `tests/reference/fixtures/html/`. Each fixture exercises a different structure (simple sections, nested subsections, sections with headings, sections with cross-references). Parser tests run against the fixtures only. They must not hit the live GhaLII site.

7.2 Unit tests for the resolver

Build a synthetic registry with three Acts and a fake Constitution under `tests/reference/fixtures/registry/`. Test every citation pattern listed in section 6.3 against this synthetic registry. Include negative tests (citations that should not resolve, citations that should be ambiguous).

7.3 Integration test, gated

One integration test that hits the live GhaLII site, fetches one specific Act (the Cybersecurity Act 2020, Act 1038, which is small and stable), and verifies the full mirror-and-parse pipeline produces a valid `LegalDocument` with at least 60 sections. Gated behind `pytest -m integration` so it does not run on every commit.

7.4 Coverage target

90 percent line coverage on `parser_xml.py`, `parser_html.py`, and `resolver.py`. 75

percent on `mirror.py` and `ghalii_client.py` (the I/O-heavy modules).

8. Dependencies

Pin minor versions in `pyproject.toml`. Use `uv` for environment management if Codex supports it, otherwise `pip` with a `requirements.txt`.

```
python = "^3.11"
requests = "^2.31"
beautifulsoup4 = "^4.12"
lxml = "^5.1"
pydantic = "^2.6"
pyparsing = "^3.1"
pyyaml = "^6.0"
rich = "^13.7"          # For CLI output
typer = "^0.9"          # For CLI structure

[dev-dependencies]
pytest = "^8.0"
pytest-cov = "^4.1"
pytest-mock = "^3.12"
ruff = "^0.3"
mypy = "^1.8"
```

No LLM dependencies in this spec. The mirror and resolver are deterministic. LLM calls come in later specs.

9. Coding standards

These are repo-wide rules. Codex must follow them under this spec.

- Every function under 30 lines. Every module under 150 lines. Split when it grows.
- Type hints on every public function. `mypy --strict` must pass on `src/lexlens/reference/`.
- One function does one thing. Loading, parsing, writing, validating, all separate.
- No hidden state. Functions take inputs as arguments and return outputs.
- No global configuration. Pass config objects explicitly.

- Logging through Python's `logging` module, configured once in `src/lexlens/logging.py`. Never use `print`.
 - Docstrings in Google style. One sentence summary, then Args, Returns, Raises if relevant. No marketing language.
 - Code comments explain why, not what. The code already says what.
-

10. Acceptance criteria

The work is done when all of these are true. Codex must run through this list before claiming completion.

1. `python scripts/mirror_ghalii.py --target constitution` downloads, parses, and writes the Constitution. The output passes a structural sanity check (at least 290 articles, every article has non-empty text).
2. `python scripts/mirror_ghalii.py --target act --number 769` downloads and parses the National Communications Authority Act, 2008. Output passes a structural sanity check (at least 80 sections).
3. `python scripts/mirror_ghalii.py --target all --dry-run` lists every current Act on GhaLII without making more than the index requests needed to enumerate them.
4. `python scripts/mirror_ghalii.py --target all` completes successfully on a fresh run. Total runtime under 30 minutes on a normal connection. No more than five `parse_failed: true` entries in the registry.
5. A second run with no upstream changes makes zero document fetch requests and completes in under 5 seconds.
6. Every citation pattern in section 6.3 has at least one passing test.
7. `pytest tests/reference/ -m "not integration"` runs in under 10 seconds with 90 percent coverage on parser and resolver modules.
8. `pytest tests/reference/ -m integration` passes against the live GhaLII site.
9. `mypy --strict src/lexlens/reference/` returns no errors.
10. `ruff check src/lexlens/reference/` returns no errors.
11. The registry file at `docs/reference/ghana_acts/registry.json` is valid JSON, conforms to the `RegistryEntry` schema, and contains an entry for every directory under

12. A short README at `src/lexlens/reference/README.md` documents the CLI, the resolver API, and how to add new aliases.
-

11. What Codex should ask before starting

If any of these are unclear, ask before writing code. Better to clarify than to rebuild.

- Owura's preferred contact email for the user-agent string.
 - Whether to commit the mirrored content to git (default yes, but the full mirror could be hundreds of megabytes; if so, use git-lfs or exclude `raw.xml` files from git and provide a fetch script).
 - Whether the existing repo already has a logging setup, a Pydantic schemas module, or a CLI framework that Codex should plug into rather than create.
 - Whether the integration test should target Act 1038 as specified or a different Act for stability.
-

12. What this spec deliberately does not include

These are out of scope and should not be built under this ticket, even if Codex thinks they would help.

- The bill parser. Bills are not Acts. The bill parser handles a different structure and lives under `src/lexlens/parsing/`. Separate spec.
 - The risk classifier or constitutional check. Those use the parsed output of this spec but are not built here.
 - Any LLM calls. The mirror and resolver are fully deterministic.
 - Any frontend or API. Pure library and CLI.
 - Diffing between Act versions. Useful eventually, not v1.
 - Resolution of references inside the bodies of Acts (Act 769 referencing Act 462). The resolver as specified handles citations passed to it, not links inside mirrored content. That is a v2 feature.
-

End of build spec 01. Next spec, 02, will cover the bill parser and clause splitter. Do not begin work on 02 until 01 is merged and tested.